

KasparAI

Checkout Intelligence Agent

TRACK 2 · TEAM SEVEN UP

KasparAI

Checkout Intelligence Agent

PRODUCT DOCUMENTATION

Reducing Cart Abandonment Through Conversational AI

Section 2

Executive Summary

KasparAI is an AI-powered checkout recovery platform built to solve one of e-commerce's most persistent revenue leaks: cart abandonment. At its core, KasparAI monitors shopper behavior in real time during the checkout journey, detects moments of hesitation or confusion, and deploys a conversational AI agent — named Kaspar — to intervene at precisely the right moment with the right message.

Unlike static popups or generic discount codes, KasparAI classifies the specific reason a shopper is hesitating — whether it is price sensitivity, shipping confusion, coupon failure, trust anxiety, or exit intent — and responds with a targeted, human-like conversation that addresses the root cause. This contextual precision is what separates KasparAI from existing solutions.

Metric	Value / Description
Problem Addressed	E-commerce cart abandonment (~70% industry average)
AI Intervention Trigger	Friction score $\geq$ 50 out of 100
Hesitation Categories	7 (price, shipping, coupon, exit, payment, trust, other)
AI Confidence Threshold	> 0.55 before triggering intervention
Discount Codes Supported	SAVE10 (10%), WELCOME15 (15%), FREESHIP
Fallback Mode	Full rule-based engine when OpenAI key is absent
Demo Pages	/, /store, /checkout, /dashboard

Section 3

Problem Statement

Cart abandonment is the single largest source of revenue loss in e-commerce. Globally, approximately seven out of every ten shoppers who add items to their cart never complete the purchase. This represents trillions of dollars in lost revenue annually across the industry. While merchants invest heavily in acquisition — paid ads, SEO, influencer campaigns — they invest far less in retaining shoppers who have already expressed buying intent by reaching the checkout stage.

The Core Challenge

Shoppers abandon checkout for a variety of specific, addressable reasons. However, most merchants treat abandonment as a single monolithic problem and respond with blunt instruments: a generic discount popup, a retargeting ad the next day, or an abandoned cart email hours later. By the time these interventions reach the shopper, the moment of decision has already passed. The shopper has either bought elsewhere, forgotten the purchase, or changed their mind entirely.

Root Cause	% of Abandonments (Industry Est.)	Current Solutions Fail Because
Unexpected shipping costs	~48%	Shown only at final step; too late
Forced account creation	~24%	UX problem; not addressed in checkout
Complicated checkout	~22%	Too many fields; poor mobile UX
Payment security concerns	~18%	No reassurance at point of hesitation
Coupon not working	~8%	No real-time help resolving the issue
Price / value uncertainty	~16%	No contextual offers at decision moment

Section 4

Why This Problem Matters

Cart abandonment is not a marginal issue — it is the central conversion problem for virtually every online retailer. The stakes are especially high because the shopper has already demonstrated intent. They browsed, selected, and began the checkout process. Losing them at this stage is far more costly than failing to attract them in the first place, because acquisition cost has already been paid.

Dimension	Impact
Revenue Loss	Industry estimates place recoverable cart abandonment revenue in the hundreds of billions annually
Acquisition Cost Wasted	If CAC is \$25 and 7 of 10 checkout visitors abandon, effective CPA exceeds \$80
Competitive Disadvantage	Merchants without recovery tooling lose sales to competitors who have them
Customer Trust	A poor checkout experience permanently damages brand perception
Merchant Stress	Without visibility into why shoppers leave, merchants cannot improve conversion systematically

KasparAI addresses this problem at the exact moment it matters: while the shopper is still on the page, still engaged, and still reachable by a conversational intervention that feels helpful rather than intrusive.

Section 5

Target Users

5.1 Buyers (Shoppers)

Attribute	Description
Who they are	Online shoppers mid-way through a purchase on a merchant's e-commerce store
Primary pain	Confusion, uncertainty, or friction that interrupts their purchase flow
What they need	Immediate, relevant answers — not generic popups or delayed emails
Interaction mode	Real-time chat with KasparAI during the checkout session
Expectation	Feels like helpful customer support, not a hard sales pitch
Outcome when served well	Completes purchase with confidence; feels positively toward the brand

5.2 Merchants (Store Owners)

Attribute	Description
Who they are	E-commerce operators running Shopify stores or similar platforms
Primary pain	High cart abandonment rate with no real-time recovery mechanism
What they need	Automated recovery that works without constant manual intervention
Interaction mode	Dashboard analytics showing recovered revenue, interventions, conversion lift
Tech comfort	Comfortable with API integrations; expects plug-and-play deployment
Outcome when served well	Measurable reduction in abandonment rate; recoverable revenue added to monthly GMV

Section 6

Current Market / Checkout Problems

Problem	Why Existing Solutions Fall Short
Generic discount popups	Not contextual — a 10% popup shown to everyone ignores the real hesitation reason and trains shoppers to wait for discounts
Abandoned cart emails	Arrive hours or days later when the shopper's intent has cooled; open rates are low
Live chat widgets	Require human agents; expensive to staff; slow to respond; unavailable 24/7
Static FAQ pages	Passive — shoppers must find and navigate them; do not address in-the-moment confusion
Exit-intent popups	Fire too late (only at exit); one-size-fits-all message; often dismissed immediately
A/B testing checkout UX	Slow iteration cycle; cannot respond to individual shopper behavior in real time
Retargeting ads	High cost; competitive noise; shopper may have already purchased elsewhere

The gap in the market is a solution that is real-time, contextual, personalized, and fully automated — operating at the moment of hesitation rather than before or after it. KasparAI fills this gap precisely.

Section 7

Proposed Solution

KasparAI is an embedded checkout intelligence layer that monitors shopper behavior, detects friction patterns in real time, and deploys a conversational AI agent to recover the sale through targeted dialogue.

Layer	What It Does	Technology
Behavior Monitoring	Tracks idle time, shipping changes, coupon failures, exit intent, payment hesitation	Custom React hook (useCheckoutTracker)
Friction Scoring	Accumulates a 0–100 score per session based on weighted behavioral events	Server-side event processing in Express
Hesitation Classification	Identifies the primary reason for hesitation from 7 categories	GPT-4o-mini via OpenAI API + rule-based fallback
Conversational Recovery	Opens KasparAI chat widget with a friction-specific opening message	Multi-turn conversation engine with three specialized agents
Offer Generation	Creates contextual discount codes or shipping upgrades based on cart value and hesitation type	PricingAgent + ShoppingAgent
Session Persistence	Maintains full conversation and event history across the checkout session	MongoDB sessions + EventLog collection
Analytics	Tracks interventions, recovery rate, and recovered revenue for the merchant dashboard	Express aggregation API + React dashboard

Section 8

Product Goals

Goal	Success Criterion
Real-time intervention	KasparAI opens within 2 seconds of friction score crossing threshold
Contextual precision	Hesitation classification confidence above 0.55 before any intervention
Friction-specific messaging	Each of 7 hesitation types triggers a distinct, tailored opening message
Zero required API keys	Full rule-based fallback means the product works with no OpenAI credential
Merchant visibility	Dashboard shows recovered sessions, intervention count, and conversion data
Non-intrusive UX	Chat widget does not interrupt checkout flow; shopper can dismiss and resume
Graceful degradation	Any backend failure surfaces a polite fallback message; checkout never breaks

Section 9

Core Features

9.1 AI Checkout Assistant

KasparAI, the conversational agent, serves as an always-available checkout concierge. It responds to natural language questions about shipping, payment, discounts, product details, and return policies. It maintains context across multiple turns, remembers what was discussed, and adjusts its tone and offers based on the shopper's situation.

9.2 Cart Monitoring

Every change to the cart — item additions, removals, quantity adjustments — is tracked and associated with the current session. Cart modifications trigger a friction score increment, signaling potential price sensitivity or indecision. The cart state is synced to the backend on every change, ensuring the AI has an accurate picture of what the shopper is considering.

9.3 Friction Detection

The friction engine assigns weighted scores to behavioral events. Exit intent scores +30, coupon failure +25, idle checkout +20, payment errors +20, shipping method changes +15, price hovering +15, address editing +10, and cart modifications +10. When the cumulative score crosses 50, the classification engine activates. This threshold was calibrated to trigger intervention early enough to recover the sale while avoiding false positives.

9.4 Coupon Assistance

When a shopper enters an invalid coupon code, the system detects the `coupon_failed` event, increments the friction score by 25, and — if classification confirms coupon confusion — opens KasparAI with an offer to provide a valid code. The PricingAgent validates codes (SAVE10, WELCOME15, FREESHIP) and communicates the discount clearly in the chat.

9.5 Shipping Clarification

Repeated shipping method changes or address edits signal confusion about delivery options. KasparAI identifies this pattern, opens with a message that acknowledges the confusion, and proactively offers a free shipping upgrade via the FREESHIP code if applicable. It can explain delivery timelines, shipping zones, and carrier options in plain language.

9.6 Delivery Estimation

KasparAI provides estimated delivery information during the conversation, helping shoppers understand when their order will arrive. This is particularly effective for shoppers who are gift-buying or time-sensitive, converting their uncertainty about timeline into confidence to complete the purchase.

9.7 Trust Recovery

A `trust_gap` classification triggers when shoppers spend extended time on the payment section, exhibit repeated idle behavior at payment, or hover repeatedly over total-price elements. KasparAI responds with trust-building messages that highlight security certifications, return policies, and the merchant's customer satisfaction record to reduce anxiety and close the sale.

9.8 Session Recovery

If a shopper closes and reopens the browser tab during checkout, their session ID is preserved in `localStorage`. The backend reloads their cart, friction score, and conversation history, allowing KasparAI to resume the conversation where it left off. This seamless continuity prevents repeated interventions and maintains context across interruptions.

9.9 Smart Recommendations

The `ShoppingAgent` analyzes the current cart's product categories and surfaces complementary items the shopper may want to add. Recommendations appear naturally within the conversation rather than as a separate popup, making them feel helpful rather than promotional.

9.10 Context Memory

KasparAI maintains a full conversation history within each session. Every message from the shopper and every response from the agent is stored in the session's `conversationHistory` array and sent with each subsequent API call. This allows the agent to reference earlier parts of the conversation, avoid repeating offers already made, and build a coherent multi-turn interaction.

9.11 Real-Time Assistance

The entire pipeline — from behavioral event to friction score update to AI classification to chat widget opening — occurs within seconds of the triggering event. Shoppers see KasparAI appear while they are still engaged with the checkout page, maximizing the window for intervention.

9.12 Conversational Checkout

Rather than navigating a static FAQ or submitting a support ticket, shoppers can type natural questions directly into KasparAI. The agent handles ambiguous language, partial questions, and multi-topic queries. It routes complex questions to the appropriate specialized agent (`CheckoutAgent`, `PricingAgent`, `ShoppingAgent`) transparently.

9.13 Abandonment Detection

Exit intent is detected via `mouseleave` events when the cursor moves toward the browser's navigation bar. This event carries the highest friction weight (+30) and almost always triggers the classification pipeline immediately. KasparAI opens within moments of the exit intent signal to catch the shopper before they navigate away.

9.14 Dynamic Response Generation

Every KasparAI response is generated dynamically by GPT-4o-mini, using the current session context, cart state, hesitation type, and conversation history as input. This means no two interactions are identical, and the agent never sounds scripted. The fallback rule engine uses a curated set of template responses that are similarly context-aware.

9.15 Personalized Checkout Guidance

KasparAI adjusts its tone, offers, and messaging based on cart value, hesitation type, and conversation history. A shopper with a \$600 cart exhibiting price hesitation receives a 15% discount offer; a shopper with a \$150 cart in shipping confusion receives a free shipping offer. This personalization makes each interaction feel tailored rather than automated.

Section 10

User Flow

Homepage Flow

Step	Action
1	Shopper lands on the homepage (/). Hero section explains KasparAI's value proposition.
2	Navigation bar offers direct links to the Store (/store) and Checkout (/checkout) demonstration.
3	Call-to-action button routes the shopper to the product catalog to begin browsing.

Product Selection Flow

Step	Action
1	Shopper browses the store (/store), which displays 30 products across 7 categories (pulled from Shopify or the 30-item mock catalog).
2	Product cards show title, category, price, and a variant selector.
3	Clicking 'Add to Cart' invokes addToCart() in CartContext, syncing the cart to the backend via POST /api/sessions.
4	Cart icon in the navigation updates quantity count in real time.

Cart Flow

Step	Action
1	Shopper reviews cart contents — items, quantities, prices, and subtotal.
2	Quantity adjustments and item removals are tracked. Item removal triggers an exit_intent signal.
3	Shopper clicks 'Proceed to Checkout' to advance to the checkout page.

Checkout Flow

Step	Action
1	Checkout page (/checkout) renders sections: Contact Info, Shipping Address, Shipping Method, Payment Details.
2	useCheckoutTracker hook activates immediately, registering the session and beginning behavioral monitoring.
3	As the shopper fills in fields, the tracker logs section dwell times, address edits, shipping method changes, and payment field focus events.
4	Each event is POSTed to /api/events, which updates the session's friction score.

AI Recovery Flow

Step	Action
1	When friction score crosses 50, the backend calls <code>analyzeHesitation()</code> , which returns a hesitation reason (e.g., 'price hesitation').
2	The session is flagged with <code>interventionTriggered = true</code> and the classified reason.
3	The frontend polls <code>/api/intervention/:sessionId</code> and detects the trigger.
4	KasparAI chat widget opens with a friction-specific opening message (e.g., 'I noticed you have been looking at shipping options — want me to check if free shipping applies to your order?').
5	Shopper types a response. Messages are sent to <code>POST /api/chat</code> and the multi-agent engine generates a reply.
6	Conversation continues until the shopper resolves their concern or the agent emits <code>[RESOLVED:]</code> .

Purchase Completion Flow

Step	Action
1	Shopper clicks 'Place Order'. Frontend calls <code>POST /api/sessions/:sessionId/complete</code> to mark the session complete.
2	A success confirmation screen is displayed.
3	The analytics dashboard (<code>/dashboard</code>) records the recovered session, adding it to the intervention and recovery counts.

Section 11

User Personas

Attribute	Persona A — The Comparison Shopper	Persona B — The Gift Buyer	Persona C — The Coupon Hunter
Age / Profile	28–40, tech-savvy professional	25–45, buying for a family event	18–35, budget-conscious student
Cart Value	\$150–\$400	\$80–\$200	\$50–\$150
Primary Hesitation	Price vs. value uncertainty; checking competitor prices	Shipping timeline uncertainty; worried about delivery before event	Coupon code not working; hunting for a better deal
Friction Signal	Repeated price hovering + idle on payment page	Multiple shipping method changes + address edits	coupon_failed event + exit intent
KasparAI Response	10% discount offer (SAVE10) + social proof message	Delivery estimate + free shipping offer (FREESHIP)	Valid coupon code provided (WELCOME15 or SAVE10) with clear instructions
Conversion Trigger	Feels the value is confirmed with the discount	Reassured delivery will arrive in time	Gets working code and feels rewarded for persistence

Section 12

Product Design Decisions

Decision	Rationale
Chat widget over full-page overlay	A floating chat widget is dismissible and non-blocking; shoppers can continue filling checkout fields while reading the agent's message, preserving their flow
Friction score threshold at 50	Below 50, events are minor and could reflect normal browsing behavior. Above 50, the accumulation indicates genuine hesitation. The threshold was set conservatively to avoid over-triggering and causing shopper annoyance
Confidence threshold at 0.55	Lower confidence classifications are unreliable; an incorrect hesitation reason triggers the wrong message type, which can worsen conversion. The 0.55 cutoff prioritizes precision over recall
Three specialized agents	A single monolithic agent would struggle to handle pricing, product recommendations, and conversational recovery with equal depth. Specialization allows each agent to be prompted and optimized independently
Rule-based fallback	Requiring an OpenAI API key for core functionality would exclude the largest segment of merchants. The rule-based engine ensures the product works correctly out of the box for everyone
sessionId in localStorage	Persistent session IDs allow recovery even if the shopper refreshes the page or briefly navigates away, maintaining continuity without requiring login

Section 13

Scope Decisions

What Was Included

Feature	Reason for Inclusion
Real-time behavioral event tracking	Core to the product's value proposition; cannot be deferred
Friction scoring engine	Required to determine intervention timing
GPT-4o-mini hesitation classification	Provides contextual accuracy impossible with rules alone
Multi-turn conversational recovery	Single-turn responses are insufficient for complex hesitation scenarios
Three specialized AI agents	Enables depth in pricing, recommendations, and checkout guidance
Rule-based fallback	Ensures the product is functional without API credentials
Merchant analytics dashboard	Essential for merchant ROI visibility; directly tied to monetization
30-item mock product catalog	Enables a realistic demo without requiring a live Shopify connection

What Was Excluded

Feature	Reason for Exclusion
Real payment processing	Stripe integration is UI-prototype only; out of scope for hackathon demo
Live Shopify webhook integration	Shopify API is optional; mock catalog handles all demo scenarios
User accounts and authentication	Login/signup pages exist in the UI but authentication is not enforced for the demo
Multi-language support	All copy and AI prompts are in English; localization is a post-MVP feature
Email/SMS abandoned cart follow-up	Outside the real-time recovery scope; complementary but separate product
A/B testing framework	Would require additional infrastructure; not required for core demo

Section 14

Tradeoffs

Tradeoff	Choice Made	What Was Sacrificed	Why It Was Worth It
AI vs. Rule-Based	Ship both; AI primary, rules as fallback	Additional complexity in the codebase	Product works for all users regardless of API access
Polling vs. WebSockets	HTTP polling for intervention trigger	Real-time push latency (~500ms delay)	Simpler to implement reliably; polling interval is short enough for the use case
GPT-4o-mini vs. GPT-4o	GPT-4o-mini chosen	Slightly lower reasoning quality	Dramatically lower cost and latency; quality is sufficient for checkout classification tasks
Single-page checkout vs. multi-step	Multi-step with section tracking	Simpler implementation of single-page	Multi-step creates more discrete dwell-time signals, improving friction detection accuracy
Inline recommendations vs. separate page	Inline in chat	Dedicated product recommendation page	Keeps shopper on the checkout page; reduces navigation abandonment risk

Section 15

UX Decisions

UX Decision	Details
Widget positioning	Bottom-right floating widget — standard convention for chat; does not obscure form fields in standard desktop or mobile viewport
Dismiss behavior	Shopper can close KasparAI at any time; the widget does not reopen automatically during the same session once dismissed
Opening message	Each of the 7 hesitation types has a distinct, non-generic opening message. Messages acknowledge the specific behavior observed without feeling surveillance-like
Agent name and persona	Named 'Kaspar' — approachable, distinctive, memorable; not a generic 'Support Bot' or 'Assistant'
Message timing	KasparAI appears approximately 1–2 seconds after the friction threshold is crossed, giving the shopper a moment to continue naturally before the intervention
Typing indicator	Kaspar shows a typing animation while the API call is in progress, setting realistic expectations for response time
Offer presentation	Discount codes are displayed prominently in the chat, formatted as copyable inline code blocks for easy application
Mobile responsiveness	Chat widget adapts to narrow viewports using Tailwind's responsive utilities; full keyboard accessibility maintained on mobile

Section 16

AI Usage Strategy

AI Component	Model	Purpose	Fallback
Hesitation Classifier	GPT-4o-mini	Analyzes session events to determine primary hesitation reason with confidence score	Rule-based event count mapping
CheckoutAgent	GPT-4o-mini	Generates persuasive checkout guidance, section navigation analysis, trust-building copy	Template-based persuasion strings
PricingAgent	Deterministic	Generates discount offers based on cart total thresholds; validates coupon codes	N/A — fully deterministic logic
ShoppingAgent	Deterministic + GPT	Category-based product recommendations; natural language product search	Array filter on product catalog
Recovery Conversation	GPT-4o-mini	Multi-turn dialogue engine combining all three agents to recover the sale	Rule-based response templates by hesitation type

All AI calls include the full session context (cart items, cart total, friction score, hesitation type, conversation history) in the system prompt. The system prompt instructs the model to emit [RESOLVED:] when the shopper accepts an offer, allowing the backend to mark the session as complete programmatically. Response validation strips any unexpected formatting before returning to the frontend.

Section 17

Failure Recovery Experience

Failure Scenario	User Experience	Technical Handling
OpenAI API unavailable	KasparAI responds using rule-based templates; shopper sees no difference in response quality for common scenarios	analyzeHesitation() catches API errors and routes to runFallbackRules()
MongoDB connection failure	Backend returns 500; frontend surfaces a friendly 'We're having trouble connecting' message; checkout form remains fully functional	Express error middleware catches Mongoose errors
Network timeout on chat	Chat shows 'Something went wrong — try again' with a retry button; conversation history is preserved	Axios catch block in frontend chat handler
Invalid session ID	Backend returns 404; frontend generates a new session and reinitializes tracking seamlessly	Session lookup failure triggers new session creation
Classification returns 'other' or low confidence	No intervention is triggered; friction score is partially reset to 25 to prevent repeated low-confidence triggers	analyzeHesitation() returns null; server resets score

Section 18

Business Impact

Impact Area	Description
Revenue Recovery	Each successfully recovered session represents a sale that would otherwise have been lost. For merchants with high average order values, even a single recovered session per day represents thousands of dollars monthly
Conversion Rate Lift	By addressing specific hesitation reasons rather than generic friction, KasparAI targets the marginal converter — the shopper most likely to be recovered — maximizing the lift per intervention
Customer Experience	A helpful, contextual AI assistant during checkout creates a positive brand association, increasing the probability of repeat purchase and word-of-mouth referral
Operational Efficiency	Fully automated recovery requires no human agent staffing; operates 24/7 with zero incremental labor cost per intervention
Merchant Insight	The dashboard surfaces which hesitation types are most common for a given store, enabling merchants to make structural improvements to their checkout UX over time
Data Network Effects	Every session enriches the classification model's training signal, improving accuracy at scale

Section 19

Competitive Advantage

Advantage	KasparAI	Generic Tools
Intervention Timing	Real-time; triggers while shopper is on-page based on behavioral signal	Delayed; fires at exit only or post-abandonment
Contextual Accuracy	7-category hesitation classification with confidence scoring	One-size-fits-all discount popup
Conversational Depth	Multi-turn AI conversation with memory and specialized agents	Static message; no follow-up
Merchant Visibility	Full session analytics with hesitation type breakdown	Email open/click rates only
Fallback Reliability	Full rule-based engine; works without any API key	Depends entirely on external API
Offer Personalization	Cart-value-aware discount selection; shipping-specific offers	Same discount code for all shoppers

Section 20

Future Improvements

Improvement	Description	Priority
Live Shopify webhook integration	Replace polling-based event collection with real-time Shopify webhooks for cart and checkout events	High
Voice interface	Add a voice-mode option for KasparAI, allowing shoppers to speak their questions during checkout	Medium
Multi-language support	Localize all agent prompts and UI copy for the top 10 e-commerce languages	High
A/B testing framework	Test different intervention thresholds, opening messages, and offer types to optimize conversion lift per segment	Medium
Stripe payment integration	Replace UI-prototype payment fields with real Stripe Checkout Elements	High
Real-time analytics wiring	Connect the dashboard to live /api/analytics data rather than mock chart data	High
Proactive recommendation engine	Use purchase history and browsing data to recommend products before hesitation occurs	Low
Merchant configuration UI	Allow merchants to customize friction thresholds, offer amounts, and opening messages without code changes	Medium
Mobile app SDK	Package the tracker and chat widget as a React Native SDK for mobile commerce apps	Low

Section 21

Conclusion

KasparAI represents a fundamental rethinking of how e-commerce merchants respond to cart abandonment. Rather than treating abandonment as an after-the-fact problem to be addressed with retargeting campaigns and follow-up emails, KasparAI intervenes in real time, at the moment of hesitation, with a conversational AI agent that addresses the specific reason a shopper is considering leaving.

The product is built on a foundation of behavioral signal collection, weighted friction scoring, AI-powered hesitation classification, and multi-turn conversational recovery. It is designed to work without any external API credentials, making it immediately accessible to all merchants, while unlocking significantly higher contextual accuracy when an OpenAI key is provided.

Team Seven Up built KasparAI as a complete, functional product — from the behavioral tracking hook to the merchant analytics dashboard — demonstrating both the technical depth and product clarity required to take this from a hackathon submission to a production-grade SaaS tool. The infrastructure is in place. The recovery conversations are live. Kaspar is ready to help shoppers check out.

Team	Track	Submission
Seven Up	Track Number 2	KasparAI — Checkout Intelligence Agent